BÀI TẬP LÝ THUYẾT
TÌM KIẾM, CSP VÀ GAME TRONG AI

Họ và tên	Trương Thị Huyền Trang
MSSV / Lớp	3123410385

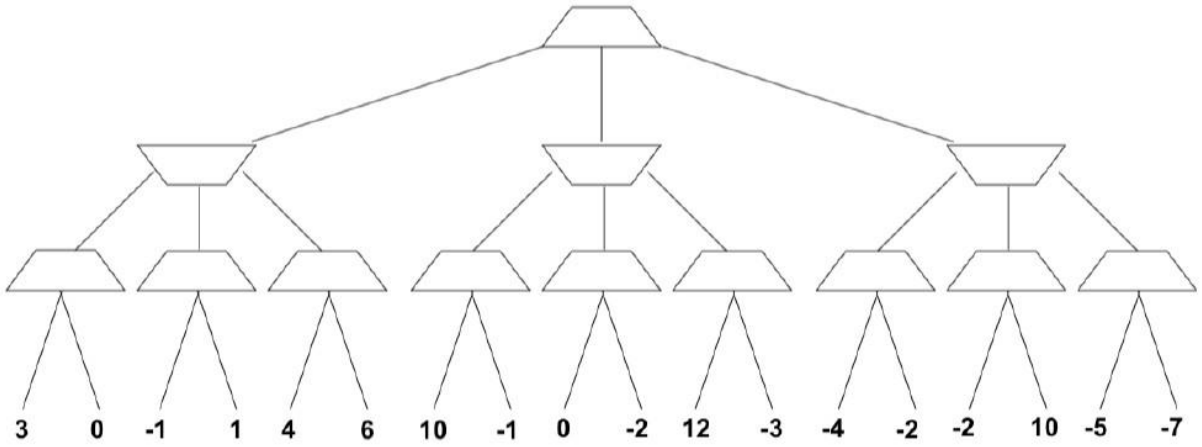
Mục tiêu học tập

- Thực hiện Minimax và Alpha-Beta Pruning, xác định giá trị các nút và các nhánh bị cắt tỉa.
- Phân tích cây trò chơi có nút ngẫu nhiên (chance nodes) và khả năng cắt tỉa.
- Vận dụng Forward Checking, Arc Consistency và suy luận trên đồ thị ràng buộc CSP.
- Giải thích tính đầy đủ, tối ưu của các thuật toán tìm kiếm và tác động khi chi phí cạnh thay đổi.
- Phân tích tác tử, khả năng quan sát của môi trường và tính hợp lý của hành vi.

Hướng dẫn: Trả lời các câu hỏi bằng lập luận ngắn gọn, trình bày các bước tính toán khi cần. Đối với câu có hình, có thể điền trực tiếp vào hình hoặc bổ sung bảng tính.

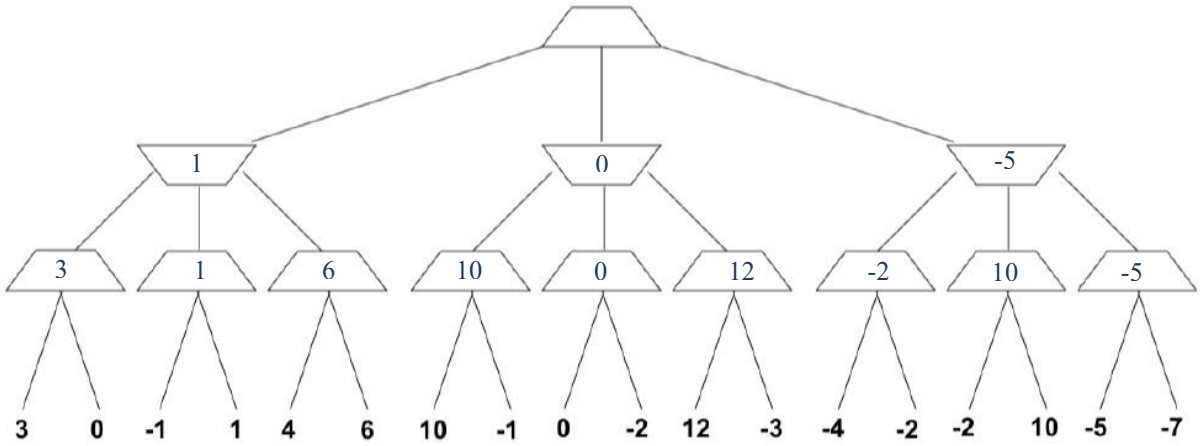
Bài 1. Minimax và Alpha-Beta Pruning

Xét cây trò chơi tổng bằng không trong Hình 1. Nút gốc là MAX, tầng tiếp theo là MIN và tầng kế tiếp là MAX. Hai người chơi đều hành động tối ưu. Các nút con được xét từ trái sang phải.



Hình 1. Cây trò chơi dùng cho Minimax và Alpha-Beta (nguồn: rec3.pdf, trang 1)

a) Áp dụng Minimax, tính giá trị mỗi nút trung gian trong cây.

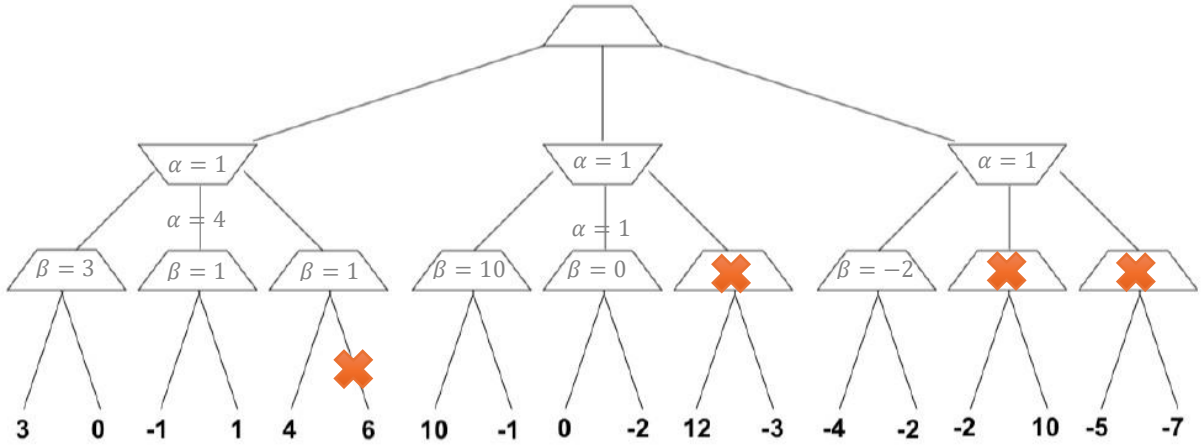


b) MAX nên chọn nhánh nào từ nút gốc? Giá trị trò chơi là bao nhiêu?

Max(1,0,-5) = 1 => Max chọn nhánh trái . Giá trị trò chơi = 1

c) Áp dụng Alpha-Beta Pruning theo thứ tự trái sang phải. Ghi α, β ở các nút đã thăm; đánh dấu X các nhánh bị cắt tỉa.

Ban đầu tại gốc: $\alpha = -\infty$; $\beta = +\infty$



d) Liệt kê các nút lá không cần thăm. So sánh số nút lá được đánh giá bởi Minimax và Alpha-Beta.

Các nút lá không cần thăm: 6, 12, -3, -2, 10, -5, -7 (7 nút)

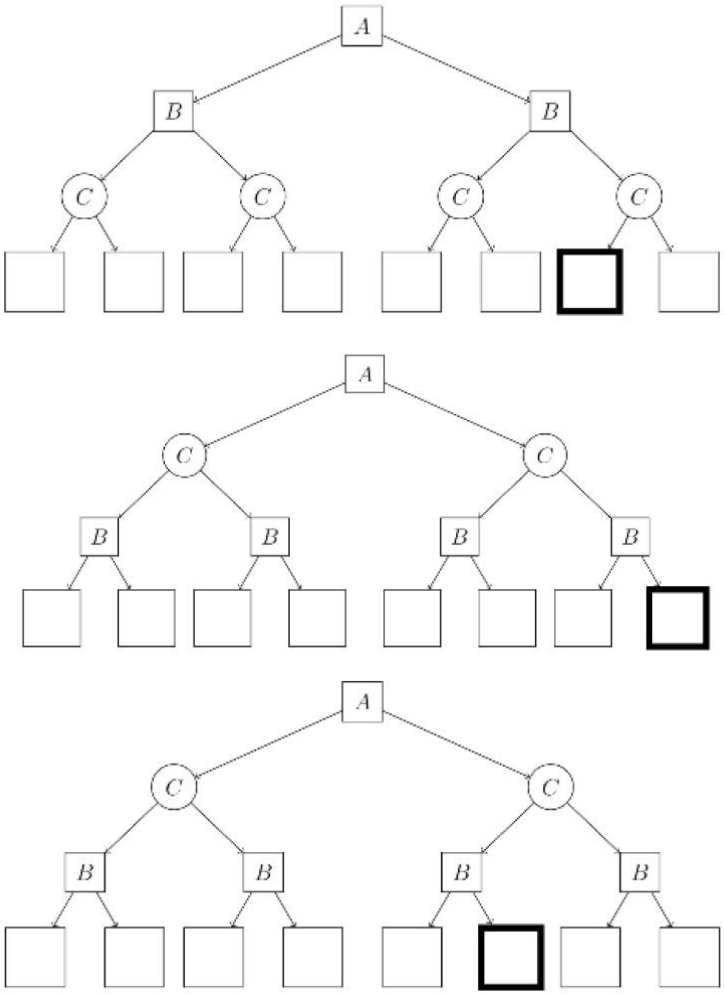
Minimax phải xét 18 nút lá.

Alpha-Beta chỉ xét 18 – 7 = 11 nút lá

⇨ Tích kiệm 7 lần đánh giá

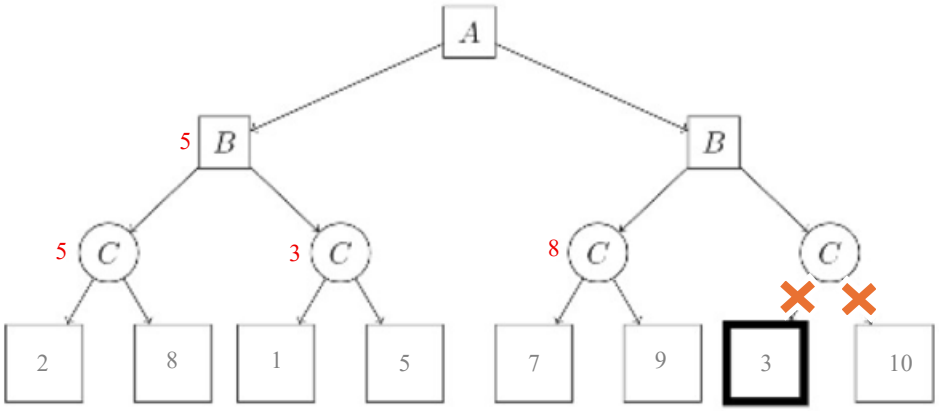
Bài 2. Alpha-Beta trong cây có nút ngẫu nhiên (Expinimax)

Xét ba cây trò chơi ở Hình 2. A là nút MIN, B là nút MAX và C là nút ngẫu nhiên (chance node). Các con của C có xác suất bằng nhau. Duyệt từ trái sang phải. Với mỗi cây, hãy gán các giá trị hữu hạn cho các nút lá để có thể cắt tỉa nút được viền đậm, hoặc ghi “Không thể” và giải thích. Không thực hiện cắt tỉa trong trường hợp bằng nhau.

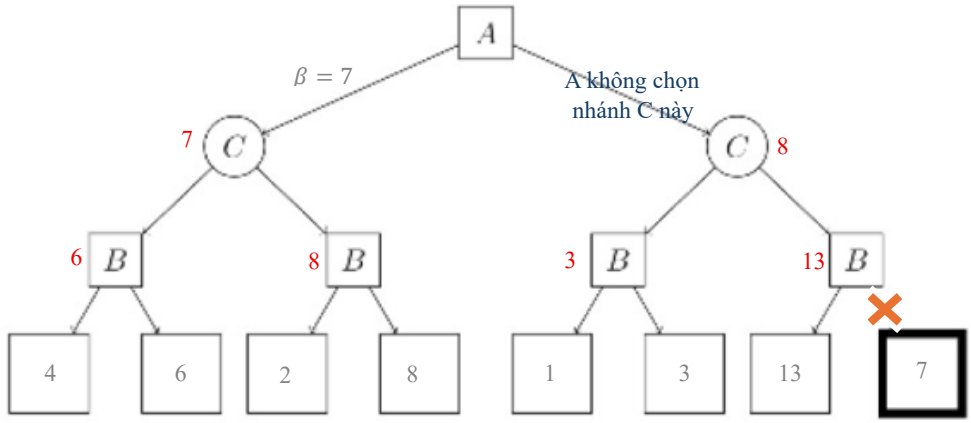


Hình 2. Ba trường hợp cắt tỉa trong cây Expinimax (nguồn: rec3.pdf, trang 3)

a) Trường hợp 1: điền các giá trị lá (nếu có thể) và chỉ ra điều kiện cắt tỉa.

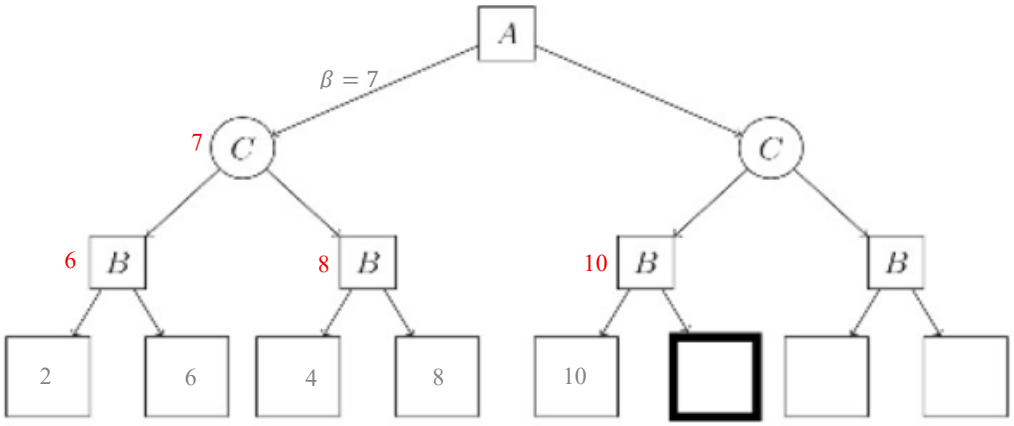


b) Trường hợp 2: điền các giá trị lá (nếu có thể) và chỉ ra điều kiện cắt tỉa.



c) Trường hợp 3: điền các giá trị lá (nế u có thể) và chỉ ra điều kiện cắt tỉa.

Giả sử chọn các lá đã được xét từ trái sang phải



Nếu ô viền đậm = 1: => B3 = max(10,1) = 10

Ô viền đậm = 20 => B3= max(10,20) = 20

Max còn lại của chance node bên phải vẫn chưa được xét. Vì giá trị của nó không có cận dưới chung đã biết, ta chưa thể chắc chắn rằng giá trị kì vọng của nhánh phải lớn hơn 7.

⇨ Không thể cắt tỉa

d) Giải thích vì sao cắt tỉa trong cây có nút ngẫu nhiên khác với Alpha-Beta trong cây Minimax.

Trong Minimax, Max và Min dùng : max, min nên chỉ cần có một giá trị vượt α *hoặc* β là có thể biết chắc nhánh đó không được chọn

Nhưng với chance node, một giá trị con lớn hay nhỏ chưa quyết định được kết quả vì các con chưa xét vẫn có thể thay đổi giá trị kì vọng.

Bài 3. Câu hỏi đúng/sai

Xét cây trò chơi có gốc MAX, giá trị Minimax là V_M. Thay mọi nút MIN bằng nút ngẫu nhiên có phân phối xác suất bất kỳ nhưng đã biết, ta nhận cây có giá trị Expectimax là V_E. Với mỗi mệnh đề, chọn Đúng/Sai và giải thích.

a) Luôn có $V_M \leq V_E$. ✔ Đúng ☐ Sai

Tại một MIN node, min(x1, x2…., xn) luôn nhỏ hơn hoặc bằng TB có trọng số $\sum_i p_i x_i$. Do đó, khi thay đối thủ xấu nhất bằng chance node: VE >= VM

b) Dùng chính sách Minimax tối ưu ở cây đã thay nút MIN bằng nút ngẫu nhiên sẽ đảm bảo lợi ích kỳ vọng ít nhất là V_M. ✔ Đúng ☐ Sai

Vì Minimax tính theo tình huống xấu nhất. Nếu đối thủ được thay bằng chance node, kết quả trung bình không thể tệ hơn giá trị nhỏ nhất mà Minimax đã dự phòng.

c) Dùng chính sách Minimax tối ưu ở cây đã thay nút MIN bằng nút ngẫu nhiên sẽ đảm bảo lợi ích kỳ vọng ít nhất là V_E. ☐ Đúng ✔ Sai

Chính sách Minimax tối ưu cho worst case, chưa chắc tối ưu cho expected value.

Xét trò chơi một người, kết quả hành động có thể không xác định trước. Người chơi nhận +1 khi thắng và −1 khi thua. Đánh dấu mọi phương pháp có thể phù hợp để mô hình hóa và giải trò chơi theo giả định cần thiết.

✔ Minimax với tiện ích trạng thái cuối +1/−1

✔ Expectimax với tiện ích trạng thái cuối +1/−1

✔ Value Iteration với phần thưởng 0 ở trạng thái trung gian và +1/−1 ở kết quả thắng/thua

☐ Không phương án nào

d) Giải thích ngắn gọn lựa chọn ở trên và những giả định cần thiết.

Minimax: phù hợp nếu coi kết quả không xác định như một đối thủ luôn chọn kết quả xấu nhất.

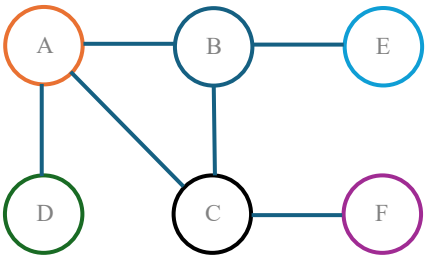
Expectimax: phù hợp nếu biết xác suất của từng kết quả.

Value Iteration: phù hợp nếu bài toán được mô hình hóa thành MDP với xác suất chuyển trạng thái đã biết; trạng thái trung gian thưởng 0, trạng thái thắng/thua thưởng +1/-1.

Bài 4. CSP với Constraint Graph và Lan truyền ràng buộc

Có 6 sinh viên A, B, C, D, E, F cần sắp chỗ ngồi. Các cặp có ràng buộc nhị phân: (A,B), (A,C), (A,D), (B,C), (B,E), (C,F).

a) Vẽ đồ thị ràng buộc tương ứng (mỗi biến là một đỉnh; mỗi ràng buộc nhị phân là một cạnh).



b) Sau khi gán giá trị cho A, miền của những biến nào có thể bị thay đổi bởi Forward Checking?

Forward Checking : xét các biến chưa gán và kề A. => B, C, D có thể bị thay đổi miền

c) Sau khi gán A và thực hiện Forward Checking, tiếp tục gán B: miền của những biến nào có thể bị thay đổi khi Forward Checking tại B?

C, E có thể bị thay đổi

d) Sau khi gán A, nếu duy trì Arc Consistency, miền của những biến nào có thể thay đổi?

AC khác Forward Checking ở chỗ việc giảm miền có thể lan truyền tiếp.

Từ A -> B, C, D.

B có thể truyền đến E; C có thể truyền đến F.

Do đó, các miền có thể bị tác động là: B, C, D, E, F.

e) Sau khi gán A và duy trì Arc Consistency, tiếp tục gán B và duy trì Arc Consistency: miền của những biến nào có thể thay đổi?

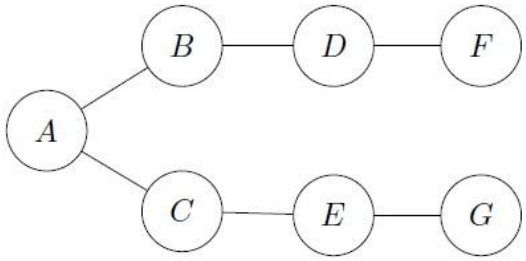
B -> C, E .

Nếu C thay đổi có thể tiếp tục lan tới F.

Vì A đã là singleton và B được chọn từ miền đã arc-consistent với A, D thông thường không bị thay đổi thêm.

Vậy: C, E, F là những miền có thể tiếp tục bị thay đổi.

Xét thuật toán mới: thực hiện Arc Consistency ban đầu và sau mỗi lần đã gán một số chẵn biến (2, 4, 6, ...). Mỗi biến có miền kích thước d. Sử dụng đồ thị ở Hình 3 để phân tích số lần backtrack trong trường hợp xấu nhất cho từng thứ tự gán; nêu rõ cách tính và các giả định.



th ràng bu c cho bài toán th t gán bi n (ngu n: rec3.pdf, trang 6)

f) Thứ tự A → B → C → D → E → F → G

Các biến luôn được mở rộng từ phần cây đã gán. Vì vậy không tạo hai vùng giá trị độc lập phải ghép lại sau này.

Worst case: 0 backtrack

g) Thứ tự G → E → C → A → B → D → F

Đây chính là đi dọc cây. Mỗi biến mới đều kề biến vừa được gán trước đó. Do AC đã được duy trì, mỗi bước có support thích hợp.

Worst case: 0 backtrack

h) Thứ tự G → F → E → D → C → B → A

Đây là trường hợp xấu nhất. G và F nằm ở 2 đầu khác nhau. E, D cũng ở 2 phía khác nhau. C, B bị ngăn cách bởi A. Chỉ tới cuối mới gán A để nối 2 phía.

Giữa 2 lần AC, tối đa có: d^2 cặp giá trị.

Trong TH xấu nhất, d^2-1 cặp có thể thất bại trước khi tìm được cặp hợp lệ.

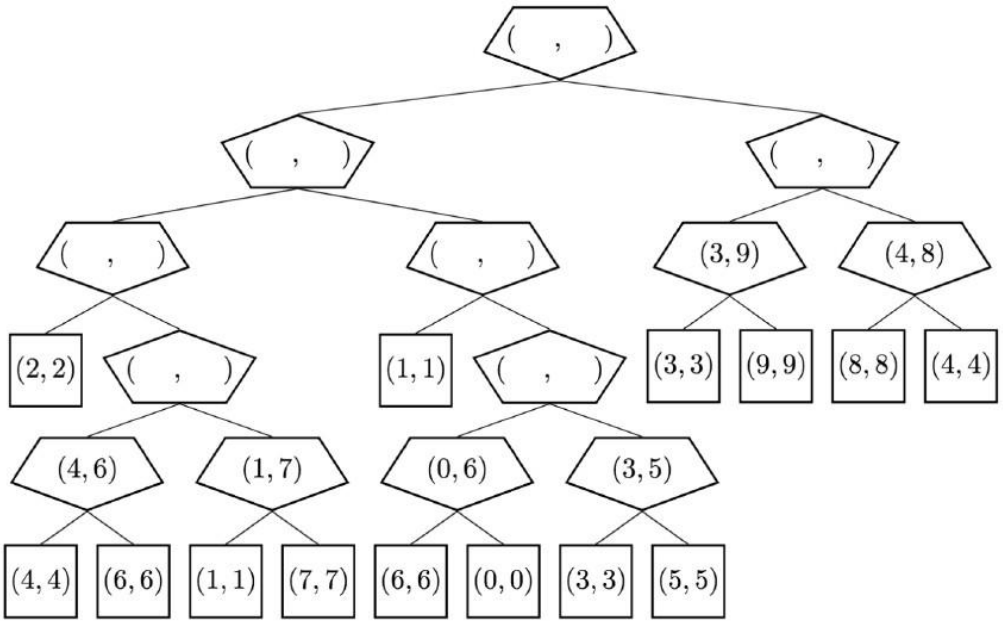
Có 3 cặp rời nhau: (G,F) , (E,D), (C, B)

Do đó, một cận xấu nhất theo cách đếm số cặp thất bại là: 3(d^2 -1).

Vì thứ tự h gán từ 2 đầu vào giữa, còn f và g phát triển theo các vùng liên kết => f=0, g=0, h có thể backtrack nhiều nhất

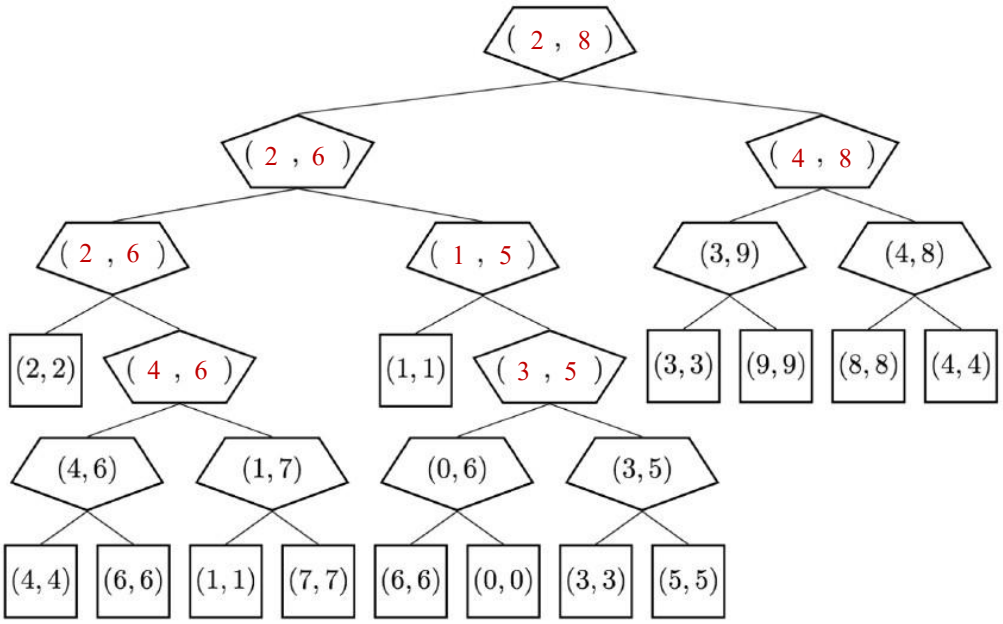
Bài 5. Cây trò chơi Pacman và Casper

Pacman tìm cách tối đa hóa tiện ích. Casper là một bóng ma thân thiện, thực sự muốn tăng tiện ích cho Pacman nhưng Pacman tưởng Casper cố giảm tiện ích. Casper biết Pacman đang hiểu nhầm. Ở mỗi nút, cặp (u, v) biểu thị lần lượt giá trị mà Pacman tin tưởng và giá trị theo hành vi thực tế của Casper. Tại lá, u = v = tiện ích cuối.



Casper

a) Điền tất cả cặp (u, v) còn thiếu trong Hình 4. Giải thích sự khác biệt giữa u và v tại nút Casper.



Pacman cho rằng Casper là đối thủ: u=min

Trong khi Casper thật sự muốn giúp => v=max

b) Theo thứ tự duyệt trái sang phải, đánh dấu các nhánh có thể không cần duyệt khi Casper tính giá trị trò chơi.

Nếu áp dụng Alpha-Beta theo giá trị Pacman tin u, thứ tự trái → phải cho phép cắt:

- nhánh (1,7);
- phần chứa (0,6) và (3,5) sau khi ngưỡng đã đủ;
- ở nhánh phải, sau khi có (3,9), theo Minimax thông thường có thể cắt nhánh (4,8).

Tuy nhiên, cắt tỉa này chỉ an toàn nếu ta chỉ cần tính u.

Nếu Casper cần chính xác cả u và v thì không thể tùy ý dùng Alpha-Beta của Minimax để cắt , vì nhánh bị bỏ có thể không ảnh hưởng u nhưng lại ảnh hưởng v

c) Nếu Pacman biết Casper thân thiện thì giá trị toàn bộ cây thay đổi như thế nào?

Lúc đó cả 2 đều muốn MAX. Do đó chỉ cần tìm tiện ích lớn nhất trong tất cả các lá.

Giá trị lớn nhất trong cây là : 9

Vì vậy, Giá trị toàn cây trở thành (9,9) thay vì (2,8)

Bài 6. Thuật toán tìm kiếm – Tính đầy đủ và tối ưu

Với mỗi thuật toán sau, xác định điều kiện để được đảm bảo đầy đủ (Complete) và tối ưu (Optimal). Phân biệt tìm kiếm trên cây và trên đồ thị nếu cần.

Thuật toán	Điều kiện đầy đủ	Điều kiện tối ưu
Breadth-First Search (BFS)	Có nếu branching factor hữu hạn và nghiệm ở độ sâu hữu hạn	Có nếu mọi bước có cùng chi phí
Depth-First Search (DFS)	Không trong cây vô hạn/có nhánh vô hạn; có nếu không gian hữu hạn và tránh lặp	Không
Iterative Deepening Search (IDS)	Có nếu branching factor hữu hạn và nghiệm ở độ sâu hữu hạn	Có nếu chi phí mỗi bước bằng nhau
A* Search	Có nếu branching hữu hạn và chi phí bước có cận dưới $\epsilon > 0$	Tree search: h admissible; graph search không reopen thường cần h consistent

Giải thích vì sao điều kiện chi phí bước và đặc tính heuristic có thể ảnh hưởng đến kết luận.

BFS và IDS tìm theo độ sâu, không tìm theo tổng chi phí. BFS có thể chọn đường 2 cạnh cost 100 dù có đường 3 cạnh cost 3 Do đó BFS/IDS chỉ bảo đảm tối ưu khi chi phí bước giống nhau.

A* dùng: f(n) = g(n) + h(n).

Để tối ưu, heuristic không nên đánh giá quá cao chi phí còn lại.

Admissible: h(n) <= h*(n)

Consistent: h(n)<=c(n, n')+h(n')

Consistency đặc biệt quan trọng với A* graph search khi không muốn phải mở lại node.

Bài 7. A* khi chi phí một cạnh thay đổi

Sau khi A* graph search tìm được đường đi tối ưu từ trạng thái đầu đến đích, chi phí cạnh X → Y thay đổi từ c thành c + n hoặc c − n, với n > 0. Với từng tình huống, hãy phân tích liệu chi phí tối ưu có thay đổi và cách cập nhật/tìm đường tối ưu mới thay vì nhất thiết chạy lại từ đầu. Khi giảm chi phí, giả sử chi phí mới vẫn hợp lệ theo mô hình tìm kiếm.

a) Chi phí tăng n; cạnh X → Y thuộc đường tối ưu cũ và đã được khám phá.

Chi phí đường cũ trở thành: C*+n

Nhưng chưa chắc đây còn là đường tối ưu vì một đường khác có thể rẻ hơn. Vì vậy chi phí tối ưu có thể tăng hoặc giữ nguyên.

Các cập nhật: Tăng cost X->Y, cập nhật g(Y); cập nhật các hậu duệ phụ thuộc vào Y, đưa các node bị ảnh hưởng trở lại OPEN, so sánh các dường thay thế.

b) Chi phí giảm n; cạnh X → Y thuộc đường tối ưu cũ và đã được khám phá.

Đường tối ưu cũ giảm n: Cnew = C*- n

Các đường khác thay đổi và các đường khác không tốt hơn C*.

Do đó đường cũ chắc chắn vẫn tối ưu. => Không cần tìm lại toàn bộ

c) Chi phí tăng n; cạnh X → Y không thuộc đường tối ưu cũ nhưng đã được khám phá.

Đường tối ưu cũ không sử dụng cạnh này nên: C* không thay đổi

Một cạnh khác bị đắt hơn nhưng cũng không thể tạo ra đường tốt hơn => Đường tối ưu cũ vẫn tối ưu.

d) Chi phí giảm n; cạnh X → Y không thuộc đường tối ưu cũ nhưng đã được khám phá.

Trường hợp này có thể tạo một đường mới tốt hơn.

Nếu sau khi giảm: g(X)+(c-n) + cost từ Y tới GOAL < C*

⇨ Đường tối ưu thay đổi

Nếu không thì vẫn dùng đường cũ => Chi phí tối ưu vẫn được giảm hoặc tối ưu.

Do X,Y đã được khám phá, ta có thể:

- cập nhật g(Y);
- reopen Y nếu cần;
- lan truyền giá trị mới tới hậu duệ.

e) Chi phí tăng n; cạnh X → Y không thuộc đường tối ưu cũ và chưa được khám phá.

Đường tối ưu cũ không dùng cạnh đó. Cạnh chưa khám phá lại còn trở nên đắt hơn. Nó không thể tạo ra một đường tốt hơn đường tối ưu cũ. C* không đổi.

f) Chi phí giảm n; cạnh X → Y không thuộc đường tối ưu cũ và chưa được khám phá.

Đây là TH nguy hiểm. Cạnh trước đây không hấp dẫn nhưng sau khi giảm có thể tạo ra đường <C*. Do cạnh chưa được khám phá, thông tin cũ có thể chưa đủ.

Cần: cập nhật cost, tiếp tục/ mở lại tìm kiếm từ phần frontier liên quan đến nếu còn lưu OPEN/CLOSED, hoặc chạy lại A* nếu trạng thái tìm kiếm cũ không còn đủ để bảo đảm tối ưu.

⇨ Chi phí tối ưu có thể giảm hoặc giữ nguyên

Bài 8. Tác tử thông minh và môi trường

Xét ba kiểu tác tử, trong đó các hàm f_A, f_B, f_C có thể ngẫu nhiên nhưng không lưu trạng thái nội bộ:

• A: act(percept) → f_A()

• B: act(percept) → f_B(percept)

• C: lưu danh sách percepts; mỗi lần act(percept) thêm percept rồi trả về f_C(percepts)

Với mỗi đặc tính môi trường sau, hãy liệt kê các lớp tác tử có thể tồn tại một hàm f để hành động hợp lý hoàn hảo trong mọi môi trường thuộc lớp đó. Giải thích ngắn gọn.

a) Quan sát đầy đủ; xác định; rời rạc; một tác tử; tĩnh.

Do quan sát đầy đủ, percept hiện tại đã chứa đủ thông tin về trạng thái.

Tác tử B có thể chọn: action=fB(percept)

C cũng có thể làm được vì C có cả percept hiện tại lẫn lịch sử.

A không đủ tổng quát vì nó bỏ qua trạng thái.

⇨ B, C

b) Quan sát một phần; xác định; rời rạc; một tác tử; tĩnh.

Percept hiện tại có thể không xác định đầy đủ trạng thái. Nhưng cần hai action khác nhau. B không phân biệt được.

C có thể dùng lịch sử: p1, p2, ..pt. để xây dựng belief về trạng thái hiện tại

⇨ C

c) Quan sát một phần; ngẫu nhiên; rời rạc; một tác tử; tĩnh.

Lại càng cần lịch sử percept để duy trì belief state. Nếu biết mô hình xác suất, C có thể tính:

$$P(State \mid percept\ history)$$

và chọn hành động tối đa hóa expected utility.

⇨ C (A, B không đủ)

d) Đúng hay sai: Có tồn tại một môi trường mà mọi tác tử đều hành động hợp lý. Nêu ví dụ hoặc phản biện.

Đúng.

Ví dụ môi trường chỉ có một hành động duy nhất. Khi đó bất kể A, B hay C trả về thế nào, nếu action duy nhất là: a. Thì a luôn là hành động tối ưu

Một ví dụ khác: môi trường có nhiều action nhưng mọi action đều cho cùng tiện ích: U(a1) = U(a2) = …=U(an). Thì bất kỳ tác tử nào chọn action nào cũng hợp lý.

— Hết —